YZM 513 – High-Performance Programming for AI

Midterm Exam — Weeks 1–6

ANSWER KEY

Name: _____ Student no.: _____ Date: _____

Time	60 minutes, closed book. A formula sheet is on the last page.
Format	25 questions, 100 points in total (4 points each).
Part A	Multiple choice, Q1–Q13. Circle *exactly one* letter.
Part B	Fill in the blank, Q14–Q25. Write one short word, number or expression on the line.

Part A – Multiple choice

[4] 1. Single-core CPU speed stopped improving much around 2005, even though transistor counts kept growing. What was the main reason?

A. Transistors could not be made any smaller.
B. Power dissipation and heat limited the clock frequency.
C. Main memory became too expensive.
D. Python introduced the GIL.

Why: The "frequency wall": clocks plateaued at 3–5 GHz, so speed-ups now come from parallelism.

[4] 2. A `cProfile` report shows that `compute_distances` takes 90% of the run time. Which tool tells you which *line* inside that function is the slow one?

A. `timeit`
B. `cProfile` (run it again)
C. `line_profiler`
D. `multiprocessing.Pool`

Why: `cProfile` works per function; `line_profiler` (`kernprof -l -v`) works per line.

[4] 3. Why do we compute `logits - logits.max(axis=1, keepdims=True)` before `np.exp` in a NumPy softmax?

A. To make the code run on the GPU.
B. To turn the Python loop into a vectorised operation.
C. To make each row sum to 1.
D. To avoid overflow in exp, without changing the result.

Why: `np.exp(1000.)` overflows to `inf` and gives `nan`. Subtracting the row maximum leaves the softmax unchanged.

[4] 4. What is the difference between Numba's `@jit` and `@njit`?

A. `@njit` raises an error if it cannot compile, instead of silently falling back to slow object mode.
B. `@njit` is always faster than `@jit`, even when both compile.
C. `@njit` turns on multi-core execution automatically.
D. `@njit` can compile Python dicts and strings; `@jit` cannot.

Why: `@njit` equals `@jit(nopython=True)`. Parallel execution needs `parallel=True` and `prange`.

[4] 5. Which situation contains a race condition?

A. Two threads only read the same NumPy array.
B. Two threads each write to their own separate array.
C. Two threads both run `counter += 1` on a shared global variable, with no lock.
D. Two threads increment a counter while holding the same `Lock`.

Why: Shared state, at least one write, and no synchronisation. Here `+=` is three steps: load, add, store.

[4] 6. You want *at most 3* threads to use the GPU at the same time. Which primitive fits?

A. `threading.Lock()`
B. `threading.Semaphore(3)`
C. `threading.Barrier(3)`
D. `threading.Event()`

Why: A semaphore allows up to N holders; a lock is the $N = 1$ case.

[4] 7. You must tokenise millions of documents with a tokenizer written in *pure Python* on an 8-core machine. Which choice gives real speed-up?

A. `ProcessPoolExecutor(max_workers=8)`
B. `ThreadPoolExecutor(max_workers=50)`
C. `asyncio.gather` over all documents
D. One thread per document with `threading.Thread`

Why: Pure-Python bytecode holds the GIL, so threads and coroutines run one at a time. Each process has its own GIL.

[4] 8. You must send 500 prompts to an LLM API server; each request mostly *waits* for the network. Which approach scales best?

A. `ProcessPoolExecutor` with 500 workers
B. 500 OS threads (about 4 GB of stacks)
C. A plain `for` loop that waits for each reply
D. `asyncio` with `await` and `gather`: one thread, 500 lightweight coroutines

Why: Coroutines cost about KB each, threads about MB each. Many concurrent I/O waits is the use case for `asyncio`.

[4] 9. `executor.submit(lambda x: x**2, 10)` fails with a `PicklingError` on a `ProcessPoolExecutor`. Why?

A. Lambdas cannot be run by the GIL.
B. Arguments and functions are sent to worker processes by pickling, and lambdas cannot be pickled.
C. `ProcessPoolExecutor` only accepts NumPy functions.
D. The pool has too few workers.

Why: Fix: define a named function with `def`.

[4] 10. A Dask job with 2000 very small tasks (about 1 ms of compute each) is *slower* than the serial code. What is the most likely cause?

A. The GIL stops Dask from using threads.
B. Dask cannot run NumPy functions.
C. The per-task overhead is as large as the compute: the granularity is too fine.
D. The network is too fast.

Why: Fix: make tasks coarser (bigger chunks), so each task computes much longer than it costs to schedule.

[4] 11. Why call `ref = ray.put(big_array)` once and pass `ref` to every task, instead of passing `big_array` itself?

A. The array is serialised once and stored in the object store; tasks receive a reference instead of a copy each.
B. It makes the array writable by all tasks at the same time.
C. It moves the array to the GPU.
D. It turns Ray tasks into actors.

Why: Principle: put big read-only data somewhere once, and pass references. Moving data is usually the real cost.

[4] 12. A matrix multiplication gains much more from a GPU than a ReLU does. What is the reason?

A. ReLU cannot be written as a CUDA kernel.
B. Tensor Cores only support matrix multiplication.
C. A GPU has fewer cores than a CPU, so simple operations are slow.
D. Matmul has high arithmetic intensity (compute-bound); ReLU has about 0.125 FLOP/byte (bandwidth-bound).

Why: For $n \times n$ float32 matmul the intensity is about $n/6$ FLOP per byte; ReLU does one operation per 8 bytes moved.

[4] 13. A student times `c = cp.dot(a, b)` (4096×4096) with `time.perf_counter()` before and after the call, and gets 0.03 ms. What went wrong?

A. The matrices were float64.
B. CuPy does not support `dot`.
C. There was no `synchronize()`, so only the asynchronous kernel *launch* was timed.
D. The arrays were still in CPU memory.

Why: Synchronise before reading the clock (or use CUDA events). Without it, every operation seems to take about 20 μs.

Part B – Fill in the blank

[4] 14. Multiplying two $n \times n$ matrices costs about $2n^3$ floating-point operations.

Why: n^2 outputs, each needs n multiplies and n adds, so $2n^3$.

[4] 15. A program has a serial fraction of 20%. By Amdahl's law, even with an unlimited number of processors the speed-up can never exceed **5** ×.

Why: $1/f_s = 1/0.2 = 5$.

[4] 16. In NumPy, `b = a[10:20]` creates a **view**, not a copy, so `b[0] = 999` also changes `a[10]`.

Why: Slices and `.T` share memory. Use `.copy()` for an independent array.

[4] 17. A `float32` array of shape 1024×1024 takes **4** MB of memory (a `float64` array of the same shape takes 8 MB).

Why: 2^{20} elements × 4 bytes = 4 MB.

[4] 18. A race condition needs three things: shared state, no synchronisation, and at least one **write** access.

Why: Many threads only reading the same data is safe.

[4] 19. Deadlock between two locks can be avoided if every thread acquires the locks in the same **order**.

Why: Lock ordering: Thread 1 holding A and wanting B while Thread 2 holds B and wants A cannot happen.

[4] 20. The GIL exists because CPython manages memory with **reference** counting, which would be corrupted if two threads updated the counts at the same time.

Why: One big lock protects all interpreter state, including every object's reference count.

[4] 21. With a serial fraction $f_s = 0.05$ and a communication fraction $f_c = 0.05$, the speed-up can never exceed **10** ×, however many workers are added.

Why: Ceiling $= 1/(f_s + f_c) = 1/0.10 = 10$.

[4] 22. In Ray, a class whose instance keeps state between calls (for example a model loaded once and used for many predictions) is called an **actor**.

Why: Stateless functions are *tasks*; stateful workers are *actors* (`@ray.remote` on a class).

[4] 23. A GPU runs its threads in groups of **32** called a *warp*. If threads of a warp take different branches, both paths are executed one after the other ("divergence").

Why: A warp is 32 threads executing one instruction in lockstep.

[4] 24. In a GPU training loop, calling `float(loss)` in every iteration forces a GPU→CPU **synchronization** and the GPU sits idle while Python waits.

Why: Accept "sync", "synchronisation" or "synchronization". Fix: accumulate the loss on the device and read it once per epoch.

[4] 25. A 4096×4096 `float32` matrix uses about **64** MB of GPU memory.

Why: $4096^2 \times 4$ bytes $= 64$ MB. Over PCIe at $\sim$12 GB/s this takes about 5 ms to transfer.

Formula sheet

Amdahl's law (f_s = serial fraction, N = number of workers):

$$S(N) = \frac{1}{f_s + \dfrac{1 - f_s}{N}} \qquad S(\infty) = \frac{1}{f_s}$$

With communication (f_c = fraction spent on overhead and data movement):

$$S_{comm}(N) = \frac{1}{f_s + f_c + \dfrac{1 - f_s - f_c}{N}} \qquad S_{comm}(\infty) = \frac{1}{f_s + f_c}$$

Matrix multiplication of two $n \times n$ matrices:

$$\text{FLOPs} = 2n^3 \qquad \text{GFLOP/s} = \frac{2n^3}{t \cdot 10^9}$$

Memory and transfer:

$$\text{bytes} = \text{elements} \times \text{bytes per element} \quad (\texttt{float32} = 4,\ \texttt{float64} = 8) \qquad t_{transfer} = \frac{\text{bytes}}{\text{bandwidth}}$$

Arithmetic intensity $= \dfrac{\text{FLOPs}}{\text{bytes moved}}$; for an $n \times n$ float32 matmul it is about $n/6$.

Units: in this exam 1 MB $= 2^{20}$ bytes (1024×1024). Typical PCIe 3.0 bandwidth: $\sim$12 GB/s.